

Pontifícia Universidade Católica do Rio Grande do Sul
Escola Politécnica

Fundamentos de Desenvolvimento de Software

Requisições e *endpoints* com parâmetros

Baseado no material do prof. Bernardo Copstein

Prof. Marcelo H. Yamaguti

Prof. Marcelo H. Yamaguti (1)

PUCRS - Politécnica

Valores de retorno de uma
requisição HTTP

- Quando se usa o Spring-Boot (como visto até aqui) o comportamento dos valores de retorno de uma requisição HTTP seguem as seguintes regras:
 - Se o valor retornado pela função que responde a requisição é um objeto Java ou uma coleção, então eles serão automaticamente serializados para um JSON.
 - Se o valor for um tipo primitivo (String, número ou booleano) , então eles serão retornados sem serem serializados.
- Em relação ao código de *status*, em caso de sucesso, este será 200.

Prof. Marcelo H. Yamaguti (2)

- Nos exemplos foram realizadas requisições GET estáticas, isto é, sem parâmetros. Para que se possa desenvolver aplicações com aplicabilidade prática, porém, é necessário que as requisições possam ser mais dinâmicas, permitindo o envio de parâmetros variáveis.
- O protocolo HTTP permite 3 formas de parametrização:
 - “Query strings” (strings de consulta)
 - “Path parameters” (parâmetros na rota)
 - “Body data” (dados no corpo da mensagem)

- No protocolo HTTP uma “query string” segue o formato:
`[path["?"<var>=<valor>["&"<var>=<valor>]]`
- Exemplo:
<http://www.servidor.br:8080/biblioteca/livrosautor?autor=Maria da Silva>
- Para tratar este tipo de requisição usa-se a anotação **@RequestParam** como no exemplo abaixo:

```
@GetMapping("/livrosautor")
public List<Livro> getLivrosDoAutor(@RequestParam(value = "autor") String
autor) {
    return livros.stream()
        .filter(livro->livro.getAutor().equals(autor))
        .toList();
}
```

- Se estiver usando o "Postman" para enviar uma requisição GET com `Query String`, especifique a requisição similar à figura:

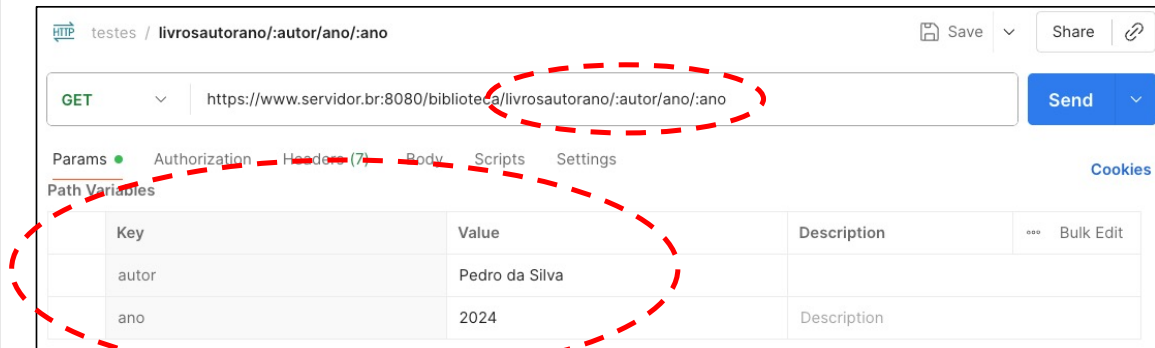


- O protocolo HTTP admite parâmetros na própria rota
- Exemplo:
<http://www.servidor.br:8080/biblioteca/livrosautorano/Pedro da Silva/ano/2024>
- Para tratar este tipo de requisição usa-se a anotação **@PathVariable** como no exemplo abaixo:

```
@GetMapping("/livrosautorano/{autor}/ano/{ano}")
public List<Livro> getLivrosDoAutor(@PathVariable(value="autor") String autor,
                                   @PathVariable(value="ano") int ano) {

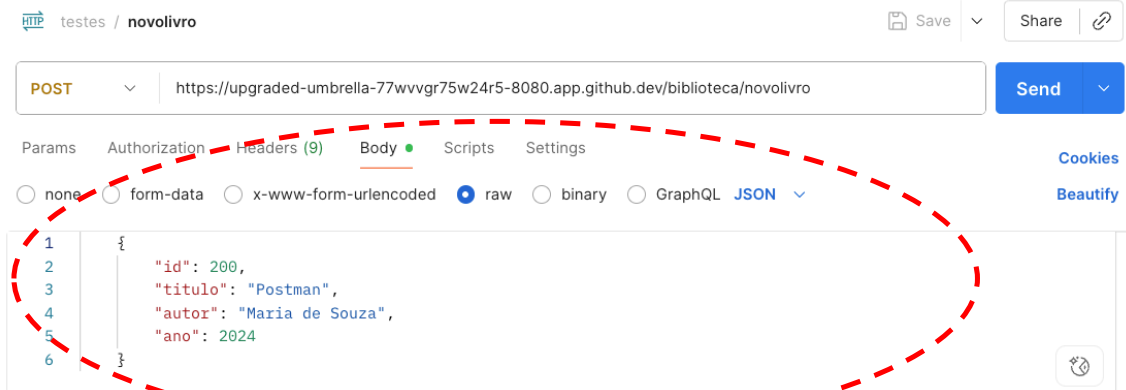
    return livros.stream()
        .filter(livro->livro.getAutor().equals(autor))
        .filter(livro->livro.getAno() == ano)
        .toList();
}
```

- Se estiver usando o "Postman" para enviar uma requisição GET com `Path parameters`, especifique a requisição similar à figura:



- O protocolo HTTP admite que se enviem dados no "corpo" (body) da mensagem.
- Normalmente este recurso é usado em requisições do tipo **POST**.
- O importante neste caso é conhecer os atributos do(s) objeto(s) que serão enviados.
- Caso contrário será difícil tratar os dados recebidos.
- Exemplo:
<http://www.servidor.br:8080/biblioteca/novolivro>

- Se estiver usando o "Postman" para enviar uma requisi  o POST com dados no corpo da mensagem, especifique o JSON com os dados similar   figura:



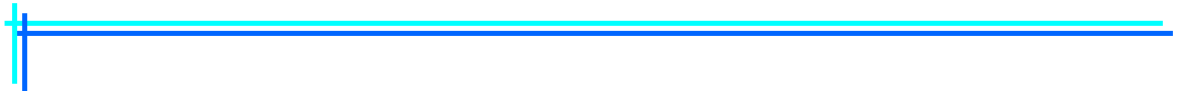
- Para receber os dados no corpo da mensagem, use a anota  o **@RequestBody** conforme o exemplo abaixo.
- O fundamental neste caso   que os atributos, do objeto que ser  enviado, correspondam aos da classe do objeto que recebe os dados.

```
@PostMapping("/novolivro")
public boolean cadastraLivroNovo(@RequestBody final Livro livro) {
    livros.add(livro);
    return true;
}
```

- Nos exemplos vistos até agora as funções que tratam os *endpoints* simplesmente retornam os valores desejados e o *framework* se encarrega da serialização e montagem da resposta HTTP.
- Em algumas situações, porém, pode ser necessário ter mais controle sobre a montagem da resposta de maneira a poder editar os códigos de retorno, cabeçalhos, etc.
- Para tanto pode-se usar a classe *ResponseEntity*.

- Usando *ResponseEntity* pode-se editar todos os atributos da mensagem resposta.
- No exemplo configura-se o *status* e o corpo da mensagem.
- Observe, também, como é feita a declaração do tipo de retorno (*ResponseEntity* é uma classe genérica).

```
@GetMapping("/livrotitulo/{titulo}")
public ResponseEntity<Livro> getLivroTitulo(@PathVariable("titulo") String titulo) {
    Livro resp = livros.stream()
        .filter(livro->livro.getTitulo().equals(titulo))
        .findFirst()
        .orElse(null);
    return ResponseEntity
        .status(HttpStatus.OK)
        .body(resp);
}
```



Pontifícia Universidade Católica do Rio Grande do Sul
Escola Politécnica

Fundamentos de Desenvolvimento de Software

Requisições e *endpoints* com parâmetros

Prof. Marcelo H. Yamaguti